

外部資料庫串接設計 (External DB Integration Design) — v3.1

本檔是專案核心商業競爭力之一：50-100 人廠 90% 都已用過鼎新 / 正航 / 叡揚 / Excel，「能不能讀我的舊資料」是採購決策第 #1 殺手。

沒這個能力，每個客戶 demo 都過不去；做得到，單一客戶就值年費 30 萬。

1. 戰略定位

1.1 為什麼這比 mobile 還重要

客戶說的話	真實意思	沒這能力會
「我們現在用鼎新」	不打算砍掉，要你能讀	demo 當場死
「我有 50 萬筆舊資料」	不想 re-key，要遷移	簽不下去
「跟 ERP 接 API」	雙向同步要會	客戶不敢買
「我跟客戶 PO 是用 Excel 對」	要會吃 CSV / Excel	業務沒武器
「我們的 SQL Server 給你連」	ODBC / SQL 端點要支援	系統整合商擋下

遷移恐懼是 ERP 採購第一殺手。「我的舊資料怎麼辦」是每位老闆都會問的問題。有了這個能力 → 「我們可以讀你的鼎新，不用砍，要遷移時 AI 自動 mapping」→ 客戶秒簽。

1.2 與「對話式 ERP」DNA 完全一致

舊資料整合不是分散精力——這正是「自然語言取代教育訓練」的延伸：

「王董打字：『鼎新的 5 月份訂單金額多少？』→ AI 自動跨 DB 查」「阿玲打字：『把鼎新的客戶搬過來』→ AI 出 Schema Mapping ConfirmCard」

Tool registry / RiskTier / ConfirmCard 全部都能複用，Phase 1 的投資直接得益。

2. 客戶常見的舊系統清單

2.1 涵蓋台灣中小製造業 ~95% 的系統

客戶常用	連接方式	MVP 優先級	難度
鼎新 Workflow ERP	SQL Server (pyodbc / pymssql)	🔥 P0	☆☆
正航 ERP	SQL Server	🔥 P0	☆☆
觀揚 ERP	REST API (OAuth2)	🟡 P1	☆☆
SAP Business One	DI API / Service Layer REST	🟡 P1	☆☆☆
Odoo	XML-RPC / REST	🟡 P1	☆☆
自家 Access / Excel	ODBC / openpyxl	🟢 PoC	☆☆
CSV 批檔 (廠商共用)	排程 watch folder	🟢 PoC	☆☆
進銷存 SaaS (雲端發票網)	REST API	🟡 P1	☆☆
金流 / 收支系統	REST API	🟡 P2	☆☆
PLC / SCADA / MES (廠房)	OPC UA / Modbus	🟡 P5+	☆☆☆☆

MVP 目標：PoC 階段做 sqlite + csv · Phase 1.5 補 SqlServer + REST API。

3. 四種連接模式



3.1 v3.0 範圍

✅ 必做：Federated Query + One-time Migration ⏸ 延後：Two-way Sync (Phase 2) / CDC (Phase 3+) — 等客戶反饋

4. 架構設計

4.1 程式碼結構

```
backend/app/integrations/connectors/  
├─ __init__.py  
├─ base.py           # Connector ABC + ConnectorMeta dataclass  
├─ registry.py       # @register_connector / get_connector  
├─ exceptions.py     # ConnectionError / TableNotFound / SchemaIncompatible  
├─ sqlite_connector.py # 內建 (PoC + 測試用)  
├─ csv_connector.py  # 內建 (PoC + watch folder 用)  
├─ postgres_connector.py # Phase 1.5  
├─ sqlserver_connector.py # Phase 1.5 (鼎新/正航)  
├─ mysql_connector.py  # Phase 1.5  
├─ rest_api_connector.py # Phase 1.5 (穀揚/SAP B1)  
└─ excel_connector.py  # Phase 1.5
```

4.2 Connector 介面契約

```
class Connector(ABC):  
    meta: ConnectorMeta # 由 @register_connector 注入  
  
    def __init__(self, config: dict):  
        """config 內含 host/port/user/password/path 等具體參數"""  
  
    @abstractmethod  
    async def test_connection(self) -> bool: ...  
  
    @abstractmethod  
    async def list_tables(self) -> list[str]: ...  
  
    @abstractmethod  
    async def query(  
        self, table: str,  
        filters: dict | None = None,  
        limit: int = 100,  
    ) -> list[dict]: ...  
  
    async def schema_of(self, table: str) -> list[dict]:  
        """回傳 [{name, type, nullable}, ...]，供 AI mapping 用。  
        預設用 query LIMIT 1 推測 schema；具體 connector 可 override。"""
```

4.3 安全防線 (4 層)

層	防線	為什麼
1	read-only by default	連接設定預設只讀，避免誤觸舊系統寫入
2	table whitelist	<code>query(table, ...)</code> 必先過 <code>list_tables()</code> 驗證，杜絕 SQL injection
3	filters 參數化	不拼接 SQL 字串，全部 prepared statement
4	稽核 log	每次外部 DB 查詢都進 audit_log，含 connection / table / filter / 結果筆數

5. AI Tool 規格 (接 Chat)

5.1 PoC 階段 (v3.0) — 3 個 read tool

Tool	RiskTier	描述
<code>list_external_connections</code>	READ	列出已設定的外部 DB 連接清單
<code>list_external_tables</code>	READ	列出某連接的所有 table
<code>query_external_db</code>	READ	跨 DB 查詢某 table，支援 filters + limit

5.2 Phase 1.5 (PoC 後續) — 加 4 個

Tool	RiskTier	描述
<code>register_external_connection_with_confirm</code>	HARD_WRITE	新增連接設定 (測 connection 通過後存 DB)
<code>delete_external_connection_with_confirm</code>	HARD_WRITE	刪連接
<code>preview_schema_mapping</code>	READ	列出外部 table 對映到 LLM-ERP domain 的建議 (AI 自動推薦)
<code>migrate_from_external_with_confirm</code>	HARD_WRITE	一次性遷移 (出 ConfirmCard 給人類確認筆數)

6. Killer 對話場景

場景 A：王董 federated query

王董：「我現在用的鼎新裡，5 月份訂單金額多少？」

AI 走 chain：

1. tool: list_external_connections → 看到「legacy_dingxin (SQL Server)」
2. tool: list_external_tables("legacy_dingxin") → 找到 OrderHeader
3. tool: query_external_db("legacy_dingxin", "OrderHeader",
filters={"order_date_gte": "2026-05-01"})
4. 加總 amount 欄位 → 回答

王董看到：「鼎新 5 月份訂單金額 \$3.2M (45 筆)。
LLM-ERP 同月 \$580K (12 筆)。
合計 \$3.78M。」

場景 B：阿玲 one-time migration

阿玲：「把鼎新的客戶搬過來」

AI 走 chain：

1. tool: list_external_tables("legacy_dingxin") → 找到 Customer
2. tool: preview_schema_mapping("legacy_dingxin", "Customer", target="Customer")

AI 建議：

鼎新.Customer.CustNo	→	LLM-ERP.Customer.code
鼎新.Customer.CustName	→	LLM-ERP.Customer.name
鼎新.Customer.Grade	→	LLM-ERP.Customer.grade
鼎新.Customer.Phone	→	LLM-ERP.Customer.phone
鼎新.Customer.Address	→	LLM-ERP.Customer.address

不對映：CreateUser, ModifyTime

3. tool: migrate_from_external_with_confirm(...) → 出 ConfirmCard
「將從鼎新匯入 124 筆客戶，欄位對映已自動偵測。
衝突策略：以 code 為唯一鍵，已存在的覆寫。
[取消] [確認匯入]」
4. 阿玲點確認 → migration 開始 → 進度條 → 完成

場景 C：小陳 CSV watch folder

小陳：「我們客戶 A 每天上午 9 點會把 PO 批檔放在 D:/orders 給我們」

AI 引導：

1. tool: register_external_connection_with_confirm(
 name="customer_a_csv", connector="csv_folder",
 config={"folder": "D:/orders"})
2. tool: schedule_external_sync(
 connection="customer_a_csv",
 schedule="*/5 * * * *", # 每 5 分鐘
 mode="ingest_new_files",
 target="sales_order")
3. ConfirmCard：「設定完成。每 5 分鐘掃 D:/orders 新 CSV 自動進 SO」

7. Schema Mapping AI 設計

7.1 自動推薦邏輯

1. 欄位名比對 (exact + LLM fuzzy)：

- CustNo ↔ code (透過 Glossary：客戶編號 = 編號 = code)
- CustName ↔ name

2. 欄位類型推測：

- VARCHAR(50) ↔ String
- DECIMAL(18,2) ↔ Float

3. Domain 約束：

- LLM-ERP customer.grade 必須 ∈ {A,B,C,D} → 預警告

4. Confidence 分數：

- 95%+ → 自動套用
- 70-95% → 出 ConfirmCard 給人類確認
- < 70% → 反問「找不到對應欄位，是否新建？」

7.2 用到 Phase 2 Glossary

docs/CONVERSATIONAL_ERP_DESIGN_ZH.md 的 Glossary 機制可直接複用：

- 「客戶」「Customer」「CustNo」「Cust」→ 都對到 LLM-ERP.Customer
- 「料件」「Part」「Material」「Item」→ 都對到 LLM-ERP.Part

8. Phase 規劃 (並行進 MVP)

Phase 1.5 (穿插 Phase 1 · 2 週並行)

#	任務	工時	對應 GAP
1.5.1	Connector framework (base + registry)	0.5d	G-501
1.5.2	SqliteConnector + CsvFolderConnector (PoC)	0.5d	G-502
1.5.3	3 個 read tool (list_conn / list_tables / query)	0.5d	G-503
1.5.4	smoke test PoC (證明跨 DB 查得到)	0.5d	G-504
1.5.5	SqlServerConnector (pyodbc)	1d	G-505
1.5.6	PostgresConnector + MySQLConnector	1d	G-506
1.5.7	RestApiConnector 抽象 + 叢揚 / SAP B1 profile	2d	G-507
1.5.8	Schema mapping AI (preview_schema_mapping tool)	2d	G-508
1.5.9	migrate_from_external_with_confirm tool (hard-write)	1.5d	G-509
1.5.10	客戶常見系統 profile (鼎新 / 正航 / 叢揚 / SAP B1)	1d	G-510

合計：10.5 工作日 (2 週並行 Phase 1)

Phase 2 (與對話智能並進) — Schema Mapping AI 補完

- AI Confidence calibration
- Domain 約束預警告
- 衝突解決策略 UI

Phase 3+ (依客戶反饋)

- Two-way Sync
- CDC Stream
- PLC / OPC UA

9. 風險與緩解

風險	影響	緩解
舊 DB 連線 driver 安裝痛苦 (特別是 SQL Server pyodbc)	客戶 IT 抓狂	Docker image 內建 driver ; 提供 install.sh 一鍵
舊 DB schema 千奇百怪	mapping AI 出錯	提供常見系統 profile (鼎新 / 正航) + 人類最終確認
舊 DB 寫入意外	砸客戶舊系統	read-only by default ; hard-write 必 ConfirmCard
SQL injection	資安事件	table whitelist + filters 參數化 (永不拼接)
效能差 (federated query 慢)	客戶體驗差	limit 預設 100 ; 大查詢走 background job + Email
客戶不會設定	導入卡關	「 Add Connection 」 UI 帶 wizard ; 連通測試即時回饋

10. 商業故事 (給銷售)

10.1 一句話

「你的鼎新不用砍。我們可以讀，AI 幫你慢慢搬，你怕沒備而後動的問題都沒了。」

10.2 三句話

「客戶採購 ERP 最怕的是『舊資料怎麼辦』。

我們直接接你的鼎新 / 正航 / 叡揚 / SAP B1，也吃 Excel 跟 CSV——AI 自動幫你 mapping 欄位、出確認卡你點一下就搬，連 SQL 都不用會寫。

90% 的對手要你『先停舊系統再導入新的』，我們是『新舊並行半年也可以』，**過渡期零風險。**」

10.3 客戶常見問答

客戶問	我們答
「鼎新可以接嗎？」	「可以。SQL Server 直連，read-only，不會動到你原系統。」
「我有 50 萬筆舊資料怎麼搬？」	「AI 自動 mapping，1 hour 跑完。期間舊系統照用，零停機。」
「會不會 corruption 我的舊資料？」	「預設只讀。要寫入要你點 ConfirmCard 確認，操作 90 秒內可 Undo。」
「我們有自家 Access」	「ODBC 接得到。沒辦法 ODBC 也吃 CSV。」
「客戶 A 每天會 sftp PO 給我們」	「設定 watch folder，每 5 分鐘自動進 SO。」

11. 與其它競品比較

競品	外部 DB 接法	LLM-ERP v3.0
SAP Business One	DI API 自己寫，貴	✓ 5 行 config 完成
Odoo	需寫 module，學習	✓ AI 對話設定
Workato / Zapier	\$\$\$ 月費、外送	✓ 內建、不送出
手寫 ETL (Talend / Pentaho)	需 IT 全職	✓ AI 取代 IT
Excel 手動匯入	慢、易錯	✓ AI 自動 mapping

12. 給開發者的指引

12.1 新增 Connector 的 5 步

1. 繼承 Connector，實作 test_connection / list_tables / query
2. 加 @register_connector(ConnectorMeta(...))
3. 在 __init__.py import 觸發註冊
4. 寫 smoke test 證明能跨 DB 讀
5. 加客戶 profile 到 docs/EXTERNAL_DB_PROFILES.md (如鼎新預設 table 對映)

12.2 安全 checklist

- ☐ read-only by default
- ☐ table 必過 `list_tables()` whitelist
- ☐ filters 用 prepared statement · 不拼接 SQL
- ☐ 結果集 `LIMIT 100` 預設
- ☐ 每次查詢進 audit_log
- ☐ sensitive 欄位 (如 password) 不回前端
- ☐ 連接設定加密儲存 (Phase 1.5)

最後更新：2026-05-15 (v3.1 戰略補強：外部 DB 串接設計) 對應 GAP：G-501 ~ G-510 對應 ROADMAP：Phase 1.5